

Experiment 'p_14_1_selection_sort_desc_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_14_1_selection_sort_desc_seq

Natural language statement: Write a method that implements the selection sort algorithm to sort an array of integers in descending order.

Method signature: p_14_1_selection_sort_desc_seq(inputs: seq<int>) returns (sorted: seq<int>)

Ensures

- forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]
- multiset(inputs) == multiset(sorted)

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that implements the selection sort algorithm to sort an array of integers
↪ in descending order.

The signature should be:

p_14_1_selection_sort_desc_seq(inputs: seq<int>) returns (sorted: seq<int>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j], ensures multiset
↪ (inputs) == multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnKCUUosQMBzJsw8wWXyhvGnHsLPW

Model: gpt-5.5-2026-04-23

Created at: 1780648094

Finish reason: stop

Usage: 328 tokens in, and 5311 tokens out

```
{
  var out: seq<int> := [];
  var rest := inputs;

  while |rest| > 0
    invariant multiset(out) + multiset(rest) == multiset(inputs)
    invariant forall i, j :: 0 <= i < j < |out| ==> out[i] >= out[j]
    invariant forall i, j :: 0 <= i < |out| && 0 <= j < |rest| ==> out[i] >= rest[j]
    decreases |rest|
  {
    var m := 0;
    var j := 1;

    while j < |rest|
      invariant 0 < |rest|
      invariant 1 <= j <= |rest|
      invariant 0 <= m < j
      invariant forall k :: 0 <= k < j ==> rest[m] >= rest[k]
      decreases |rest| - j
```

```

{
  if rest[j] > rest[m] {
    var oldM := m;
    m := j;
    assert forall k :: 0 <= k < j ==> rest[m] >= rest[k] by {
      forall k | 0 <= k < j ensures rest[m] >= rest[k] {
        assert rest[oldM] >= rest[k];
        assert rest[m] > rest[oldM];
      }
    }
  }
}

assert forall k :: 0 <= k < j + 1 ==> rest[m] >= rest[k] by {
  forall k | 0 <= k < j + 1 ensures rest[m] >= rest[k] {
    if k < j {
      assert rest[m] >= rest[k];
    } else {
      assert k == j;
      assert rest[m] >= rest[k];
    }
  }
}
j := j + 1;
}

assert forall k :: 0 <= k < |rest| ==> rest[m] >= rest[k];

var oldOut := out;
var oldRest := rest;
var x := oldRest[m];

out := oldOut + [x];
rest := oldRest[..m] + oldRest[m + 1..];

assert oldRest == oldRest[..m] + [oldRest[m]] + oldRest[m + 1..];
assert x == oldRest[m];
assert multiset(oldRest) == multiset(oldRest[..m]) + multiset([x]) + multiset(oldRest[
  ↪ m + 1..]);
assert multiset(rest) == multiset(oldRest[..m]) + multiset(oldRest[m + 1..]);
assert multiset(out) == multiset(oldOut) + multiset([x]);
assert multiset(out) + multiset(rest) == multiset(oldOut) + multiset(oldRest);
assert multiset(out) + multiset(rest) == multiset(inputs);

assert forall i, j :: 0 <= i < j < |out| ==> out[i] >= out[j] by {
  forall i, j | 0 <= i < j < |out| ensures out[i] >= out[j] {
    if j < |oldOut| {
      assert out[i] == oldOut[i];
      assert out[j] == oldOut[j];
      assert oldOut[i] >= oldOut[j];
    } else {
      assert j == |oldOut|;

```

```

    assert i < |oldOut|;
    assert out[i] == oldOut[i];
    assert out[j] == x;
    assert oldOut[i] >= oldRest[m];
  }
}
}

assert forall i, j :: 0 <= i < |out| && 0 <= j < |rest| ==> out[i] >= rest[j] by {
  forall i, j | 0 <= i < |out| && 0 <= j < |rest| ensures out[i] >= rest[j] {
    if i < |oldOut| {
      assert out[i] == oldOut[i];
      if j < m {
        assert rest[j] == oldRest[j];
        assert oldOut[i] >= oldRest[j];
      } else {
        assert j + 1 < |oldRest|;
        assert rest[j] == oldRest[j + 1];
        assert oldOut[i] >= oldRest[j + 1];
      }
    } else {
      assert i == |oldOut|;
      assert out[i] == x;
      if j < m {
        assert rest[j] == oldRest[j];
        assert x >= oldRest[j];
      } else {
        assert j + 1 < |oldRest|;
        assert rest[j] == oldRest[j + 1];
        assert x >= oldRest[j + 1];
      }
    }
  }
}
}

sorted := out;
assert |rest| == 0;
assert multiset(rest) == multiset([]);
assert multiset(sorted) == multiset(inputs);
}

```

Final Program

```

method p_14_1_selection_sort_desc_seq(inputs: seq<int>) returns (sorted: seq<int>)
  ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]
  ensures multiset(inputs) == multiset(sorted)
{
  var out: seq<int> := [];
  var rest := inputs;

```

```

while |rest| > 0
  invariant multiset(out) + multiset(rest) == multiset(inputs)
  invariant forall i, j :: 0 <= i < j < |out| ==> out[i] >= out[j]
  invariant forall i, j :: 0 <= i < |out| && 0 <= j < |rest| ==> out[i] >= rest[j]
  decreases |rest|
{
  var m := 0;
  var j := 1;

  while j < |rest|
    invariant 0 < |rest|
    invariant 1 <= j <= |rest|
    invariant 0 <= m < j
    invariant forall k :: 0 <= k < j ==> rest[m] >= rest[k]
    decreases |rest| - j
  {
    if rest[j] > rest[m] {
      var oldM := m;
      m := j;
      assert forall k :: 0 <= k < j ==> rest[m] >= rest[k] by {
        forall k | 0 <= k < j ensures rest[m] >= rest[k] {
          assert rest[oldM] >= rest[k];
          assert rest[m] > rest[oldM];
        }
      }
    }
  }

  assert forall k :: 0 <= k < j + 1 ==> rest[m] >= rest[k] by {
    forall k | 0 <= k < j + 1 ensures rest[m] >= rest[k] {
      if k < j {
        assert rest[m] >= rest[k];
      } else {
        assert k == j;
        assert rest[m] >= rest[k];
      }
    }
  }
  j := j + 1;
}

assert forall k :: 0 <= k < |rest| ==> rest[m] >= rest[k];

var oldOut := out;
var oldRest := rest;
var x := oldRest[m];

out := oldOut + [x];
rest := oldRest[..m] + oldRest[m + 1..];

assert oldRest == oldRest[..m] + [oldRest[m]] + oldRest[m + 1..];

```

```

assert x == oldRest[m];
assert multiset(oldRest) == multiset(oldRest[..m]) + multiset([x]) + multiset(oldRest[
  ↪ m + 1..]);
assert multiset(rest) == multiset(oldRest[..m]) + multiset(oldRest[m + 1..]);
assert multiset(out) == multiset(oldOut) + multiset([x]);
assert multiset(out) + multiset(rest) == multiset(oldOut) + multiset(oldRest);
assert multiset(out) + multiset(rest) == multiset(inputs);

assert forall i, j :: 0 <= i < j < |out| ==> out[i] >= out[j] by {
  forall i, j | 0 <= i < j < |out| ensures out[i] >= out[j] {
    if j < |oldOut| {
      assert out[i] == oldOut[i];
      assert out[j] == oldOut[j];
      assert oldOut[i] >= oldOut[j];
    } else {
      assert j == |oldOut|;
      assert i < |oldOut|;
      assert out[i] == oldOut[i];
      assert out[j] == x;
      assert oldOut[i] >= oldRest[m];
    }
  }
}

assert forall i, j :: 0 <= i < |out| && 0 <= j < |rest| ==> out[i] >= rest[j] by {
  forall i, j | 0 <= i < |out| && 0 <= j < |rest| ensures out[i] >= rest[j] {
    if i < |oldOut| {
      assert out[i] == oldOut[i];
      if j < m {
        assert rest[j] == oldRest[j];
        assert oldOut[i] >= oldRest[j];
      } else {
        assert j + 1 < |oldRest|;
        assert rest[j] == oldRest[j + 1];
        assert oldOut[i] >= oldRest[j + 1];
      }
    } else {
      assert i == |oldOut|;
      assert out[i] == x;
      if j < m {
        assert rest[j] == oldRest[j];
        assert x >= oldRest[j];
      } else {
        assert j + 1 < |oldRest|;
        assert rest[j] == oldRest[j + 1];
        assert x >= oldRest[j + 1];
      }
    }
  }
}
}
}
}

```

```
sorted := out;  
assert |rest| == 0;  
assert multiset(rest) == multiset([]);  
assert multiset(sorted) == multiset(inputs);  
}
```

Total Token Usage

Input tokens: 328

Output tokens: 5311

Reasoning tokens: 4095

Sum of 'total tokens': 5639

Experiment Timings

Overall Experiment started at 1780648087930, ended at 1780648227952, lasting 140022ms (140.02 seconds)

Iteration #1 started at 1780648088256, ended at 1780648227951, lasting 139695ms (139.70 seconds)